



Practica Parcial 1

1er fecha 2025

Ejercicio 1

a) (Pts.25) Defina, utilizando EBNF, la gramática para una función de C. Puede considerar (además de otros) un ejemplo como (no implemente solo el ejemplo):

```
tipo nombre_funcion (lista parámetros)
{
    instrucciones
    return xxxx (valor/variable del tipo de la función)
}
```

b) (Pts. 5) Realice el diagrama sintáctico de la gramática correspondiente.

Ejercicio 2

Sea el siguiente código en Ada, indique para todos los identificadores

a) (Pts. 5) Su tipo de ligadura con l-valor. b) (Pts. 5) Su r-valor al momento de declaración (Inicialización).

c) (Pts. 5) Tiempo de vida. d) (Pts. 5) Alcance

```
1. with text_io; use text_io;
2. Procedure Main is;
3. type arreglo_enteros is array(integer range
   <>);
4. b, m, t:integer;
5. vec:arreglo(1..100);
6. x: constant integer:=5;
7. Procedure Proc is;
   7.1. type puntero is access integer;
   7.2. vec2:arreglo(0..x);
   7.3. t: puntero;
   7.4. begin
       7.4.1. m:=4;
       7.4.2. vec2(m):= vec2(1) + vec(4);
       7.4.3. t:= new puntero;
       7.4.4. free t;
   7.5. end;
8. begin
9. m:=5;
10. Proc;
11. b:= m + 2;
12. end.
```

Realice este ejercicio sobre esta misma hoja.

Identif	L-value	R-Value	Alcance	T.V.
Main (2)	—	—	3-12	2-12
b(4)	automática	basura	5-12	2-12
m(4)	automática	basura	5-12	2-12
t(4)	automática	basura	5-7.3 8-12	2-12
vec(5)	automática	basura	6-12	2-12
x(6)	automática	basura	7-12	2-12
Proc(7)	—	—	7.1-12	7-7.5
vec2(7.2)	semiautomática	basura	7.3-7.5	7-7.5
t(7.3)	automática	nil	7.4-7.5	7-7.5
t^	dinámica	basura	7.4-7.5	7.43-7.44

Ejercicio 3

Sea el siguiente programa escrito en Pascal like, realice la ejecución del programa presentado siguiendo:
a) (Pts. 20) Cadena estática. b) (Pts. 5) Especifique cuáles son los campos que componen un registro de activación y para qué sirve cada uno de ellos.

El lenguaje evalúa expresiones de izquierda a derecha.

```
Program MiPrograma
var i: integer;
var x: integer;
var a: array[1..6] of integer;
```

Procedure Tres

```
begin
  i := i + 2;
  x := x - 1;
end
```

Procedure Dos

```
var x: integer;
Function Fun: integer;
begin
  i := i - 1;
  return x + 1;
end
begin
  x := 7;
  a[i] := Fun();
end
```

Procedure Uno

```
var i: integer;
begin
  for i:=1 to 6 begin
    a[i] := i * 2;
  end
  i := 6
  Dos();
  Tres();
  write(i);
end
```

```
begin
  i := 3;
  x := 5;
  Uno();
  write(i);
  write(x);
  for i := 1 to 6 begin
    write(a[i]);
  end
end
```

Ejercicio 4

- a) (Pts. 10) Indique dónde se almacenan las variables semi-dinámicas y cómo es el mecanismo que se utiliza para almacenar los valores de retorno en el registro de activación.
b) (Pts. 10) Indique cómo es la inicialización por defecto de las variables en lenguaje C al momento de su alocaión en memoria.
c) (Pts. 5) ¿Qué diferencias hay entre ligadura estática y dinámica? Defina el concepto de ligadura (para los distintos atributos de las variables tomando como ejemplo al lenguaje C).

Ejercicio 1

a):

$G = (N, T, S, P)$

$N = \{ \langle \text{funcionC} \rangle, \langle \text{tipo} \rangle, \langle \text{id} \rangle, \langle \text{instrucciones} \rangle, \langle \text{retorno} \rangle, \langle \text{caracter} \rangle, \langle \text{digito} \rangle, \langle \text{letra} \rangle \}$

$T = \{ "(", ")", "|", "\{", "\}" , "int", "return", "double", "string", "void", "static", "a..z", "A...Z", "0..9" \}$

$S = \langle \text{funcionC} \rangle$

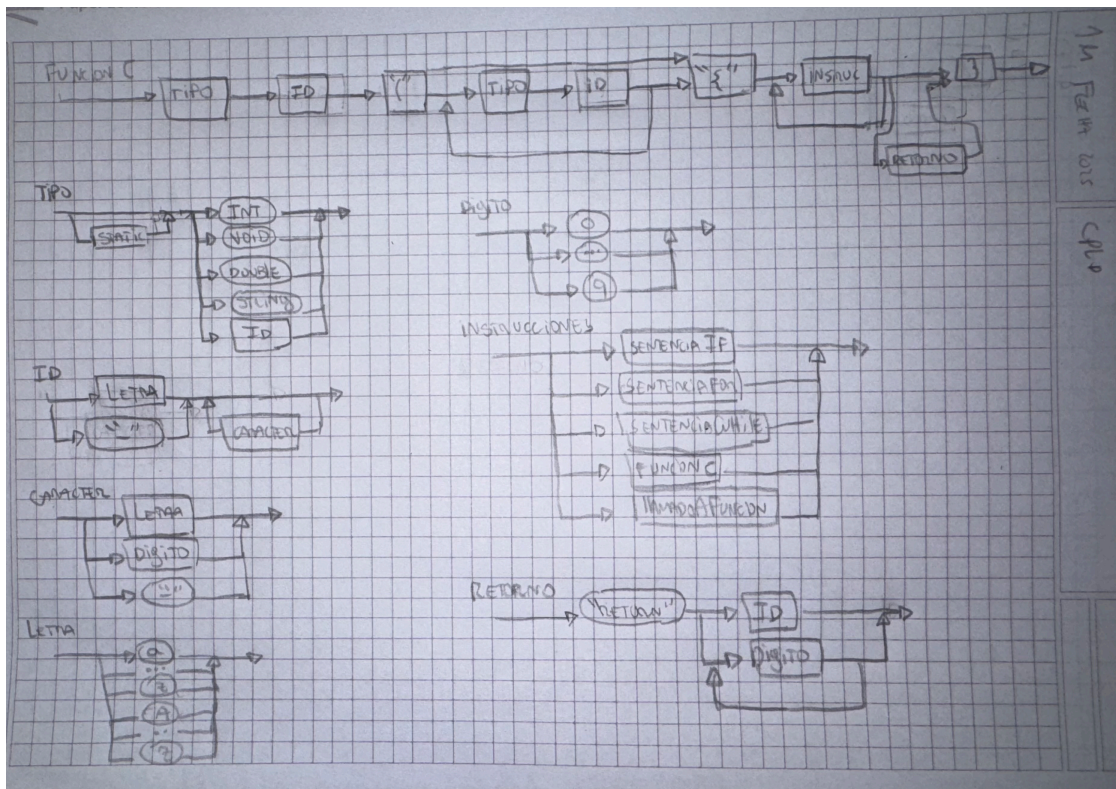
$P = \{ \langle \text{funcionC} \rangle ::= \langle \text{tipo} \rangle \langle \text{id} \rangle "(" [{"\langle \text{tipo} \rangle \langle \text{id} \rangle"}*] ")" "{" \langle i$

```

instrucciones> [<retorno>] "{"
    <tipo> ::= ["static"] ("int"|"string"|"double"|"void" |<id
>)
    <id> ::= (<letra> | "_" ) {<caracter>}*)
    <caracter> ::= (<letra>| "_" |<digito>)
    <letra> ::= (a|..|z|A|..|Z)
    <digito> ::= (0|1|..|9)
    <instrucciones> ::= (<sentenciaIf>|<sentenciaFor>|<senten
ciaWhile>|<funcionC>|
        llamadoAFuncion>|..)
    <retorno> ::= "return" (<id>|{<digito>}+)
}

```

b)



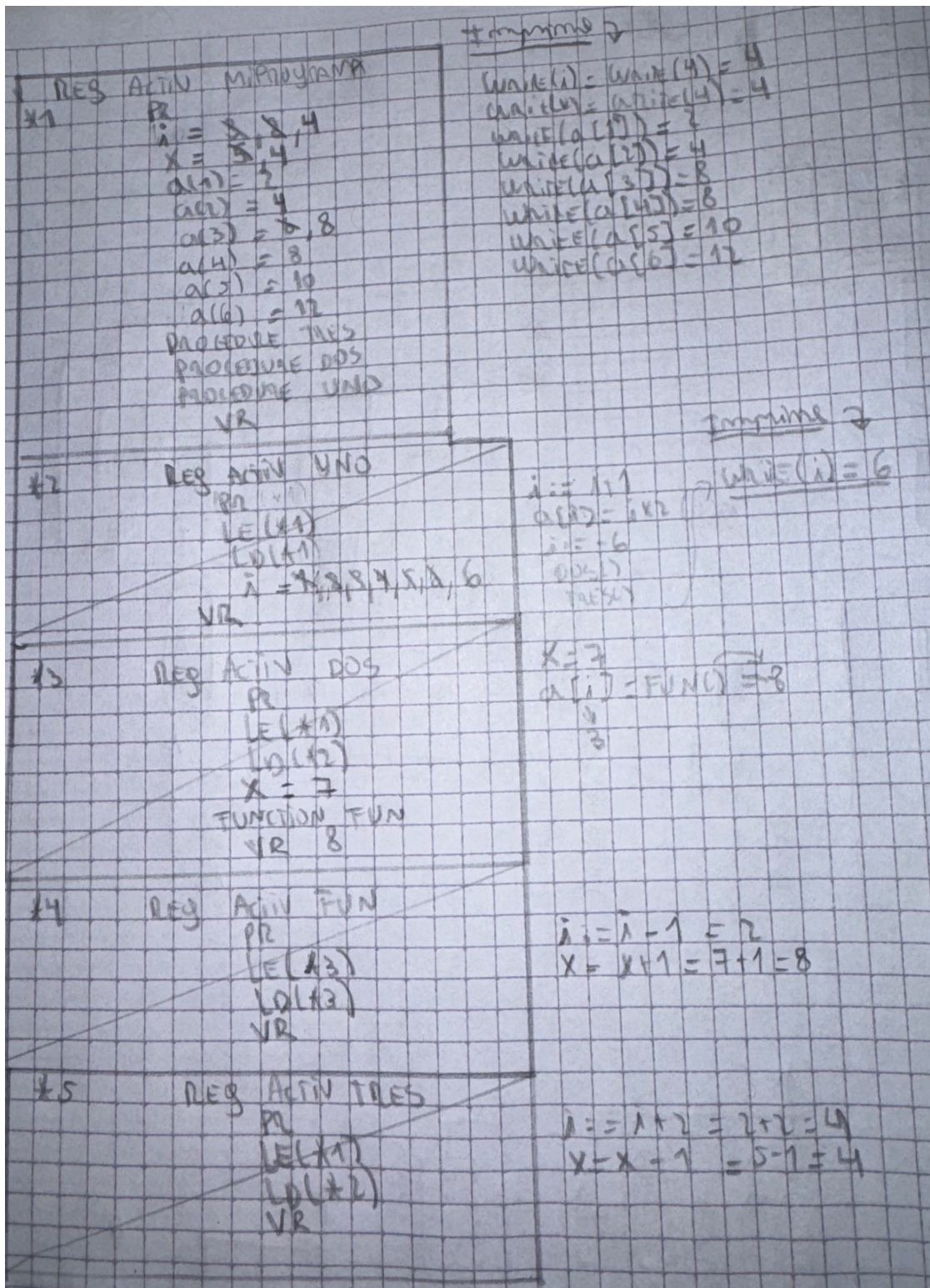
Ejercicio 2)

Ident.	Tipo	r-valor	Alcance	T.V.
Main (2)			3-12	2-12
b (4)	automatica	basura	5-12	2-12
m (4)	automatica	basura	5-12	2-12
t (4)	automatica	basura	5-7.3 → 8-12	2-12

Ident.	Tipo	r-valor	Alcance	T.V.
vec (5)	automatica	basura	6-12	2-12
x (6)	automatica	5	7-12	2-12
Proc (7)			7-7.5	7-7.5
vec2 (7.2)	semidinamica	basura	7.3-7.5	7-7.5
t (7.3)	automatica	nil	7.4-7.5	7-7.5
t^	dinamica	basura	7-4-7.5	7.4.3-7.4.4

Ejercicio 3)

a)



b) Los campos que componen un registro de activacion son:

- Registro de Activacion → Valores y atributos de un prodecimiento, main o funcion en forma de pla
- Punto de retorno → Guarda la direccion de memoria donde la ejecucion continua luego de llamar a una rutina

- Link estatico → Es un enlace al registro donde estaticamente esta contenido el registro de activacion
- Link dinamico → Es un enlace al registro que dinamicamente llamo o invoca al registro de activacion
- Variables → Son variables del reg. activacion que pueden ir cambiando de valor a lo largo de la ejecucion del programa
- Parametros → Valores de los parametros al momento de ser invocado el reg activacion
- Procedures → Son procedimientos declarados dentro del bloque del reg. activacion
- Funciones → Funciones declaradas dentro del bloque del reg. activacion
- Valor de Retorno → Se almacena el valor del llamado a una funcion dentro de la logica del bloque una vez esta termina. Este valor tiene que ser almacenado aqui ya que de otra manera se perderia y es en el registro de activacion en donde es llamada esa funcion que se almacenaria

Ejercicio 4)

a) Indique donde se almacena las variables semidinamicas y como es el mecanismo que se utiliza para almacenar los valores de retorno en el registro de activacion

-Las variables semidinamicas se almacenan en la stack dentro del R.A de la funcion o bloque donde se declara. Debido a que su tipo es estatico pero su tamaño es desconocido (ya que los limites superiores se determinan en tiempo de ejecucion) un arreglo en Ada puede tener como limite superior N, siendo N desconocido y tipado. El mecanismo que se utiliza para los valores de retorno es que si se invoca una funcion dentro de un bloque de un registro de activacion y esta termina, el valor que esta retorna es guardado en el valor de retorno del registro de activacion donde esta fue invocada. Si no se hace esto se perderia el retorno de la funcion en si una vez se desapila el registro donde se invoca

b) Indique como es la inicializacion por defecto de las variables en lenguaje C al momento de su alocaion en memoria

-Las variables globales y estaticas permiten inicializacion por default en C y dependen del tipo de variable. Por ejemplo, en el caso de los enteros, se inicializan en 0; los caracteres en blanco, etc. En cambio, las variables locales que no son ni estaticas ni globales no se inicializan por default en memoria y contienen basura al momento de alocarse en memoria

c) Que diferencias hay entre ligadura estatica y dinamica? Defina el concepto de ligadura para los distintos atributos de las variables tomando como ejemplo al lenguaje C

-Ligadura → Es el momento en el que a un atributo (identificador) de una entidad se le asocia un valor determinado.

-Ligadura estatica → Ocurre antes de la ejecucion y no se puede modificar

-Ligadura dinamica → Ocurre durante la ejecucion y puede ser modificado

-Ejemplos en C:

-Tipo → `int X;` → Se liga el tipo entero a la variable X, es ligadura estatica

-Nombre → `int X` → Se liga el identificador con el cual se va a reconocer la variable, en este caso es X, es ligadura estatica ya que no puede cambiarse

-L-valor → Se liga la direccion de memoria en ejecucion de la variable X, es ligadura dinamica

-R-valor → `x = 5 ; x = 10` → Se ligan diferentes valores de la variable x, ligadura dinamica

-Alcance → Rango en el que se va a reconocer a la variable X, es estatico aunque C permite enmascarar variables

-Tiempo de vida → Espacio de tiempo en el que una variable va a tener vida durante la ejecucion del bloque o programa donde esta contenida, es ligadura dinamica

1er fecha 2024

Ejercicio 1

a) (Pts.25) Defina, utilizando BNF, la gramática para un switch en Java. Puede considerar (además de otros) un ejemplo como:

```
switch(expresion) {  
  case valor1:  
    // bloque 1  
    break;  
  case valor2:  
    // bloque 2  
    break;  
  default:  
    // bloque default  
}
```

Realizar la mayor cantidad de combinaciones posibles, acerca de valores y tipos de datos para expresión (pueden ser valores, identificadores, etc). Los valores que definen cada rama son valores literales. **No realice sólo el ejemplo.**

b) (Pts. 5) Realice el diagrama sintáctico de la gramática correspondiente.

Ejercicio 2

Sea el siguiente código en Ada, indique para todos los identificadores

a) (Pts. 5) Su tipo de ligadura con l-valor. b) (Pts. 5) Su r-valor al momento de declaración (inicialización).
c) (Pts. 5) Tiempo de vida . d) (Pts. 5) Alcance

```
1. with text_io; use text_io;  
2. Procedure Main is;  
3.   type arreglo_int is array(integer range <>);  
4.   c, o, r: integer;  
5.   vec: arreglo(1..100);  
6.   c3: constant integer:=10;  
7.   Procedure Proc is;  
   7.1.   type pointer is access integer;  
   7.2.   vec2: arreglo(0..c3);  
   7.3.   r: pointer;  
   7.4.   begin  
       7.4.1.   o:=4;
```

Realice este ejercicio sobre esta misma hoja.

Identif.	L-value	R-Value	Alcance	T.V.
----------	---------	---------	---------	------

```

7.4.2.   vec2(o) := vec2(1) + vec(5);
7.4.3.   r := new pointer;
7.4.4.   free r;
7.5.     end;
8.   begin
9.   o := 5;
10.  Proc;
11.  c := o + 2;
12.  end.

```

Ejercicio 3

Sea el siguiente programa escrito en Pascal like, realice la ejecución del programa presentado siguiendo:
a) (Pts. 20) Cadena estática. b) (Pts. 5) Indique cuáles son los campos que componen un registro de activación y para qué sirve cada uno.

```

Program principal
Var k: integer;
Var est: boolean;
var z: integer;
Procedure puno;
Var c: integer;
Var vec: array[1..3] of integer;
Var z: integer;
Begin
  c:=1;
  z:=1;
  for k:=1 to 3 do begin
    z:=f+3;
    vec[k]:=z;
  end;
  if (est = true) then begin
    c:=z;
  end;
  write(c, z);
  For k=1 to 3 begin write(vec(k)); end;
End;

```

```

Procedure proc2;
var f: integer;
begin
  est := (k mod 2) = 0;
  f:=1+z;
  if (z=0) puno;
  write(f);
  z:=1;
end;

```

```

Función f: integer;
Begin
  z:=k;
  Return k*2;
End;

```

```

Begin
  z:=0;
  k:=3;
  est:=true;
  proc2();
  Write(k,est,z);
END.

```

Ejercicio 4

- a) (Pts. 10) Indique dónde se almacenan las variables estáticas en C y cuál es la política de inicialización de variables en C para las variables globales y estáticas.
b) (Pts. 10) Indique un ejemplo de variable donde su tiempo de vida sea mayor que su alcance. El ejemplo debe ser claro de acuerdo a lo visto en la explicación en cuanto a la alocaión en memoria, no solo considerando los números de línea en el código. El ejemplo puede ser en pseudocódigo (emulando pascal like, por ejemplo)
c) (Pts. 5) Sea la siguiente afirmación: "un lenguaje interpretado sólo detecta errores semánticos, ya que se chequea todo en ejecución". Indique si la misma es verdadera o no, dando una justificación apropiada.

Ejercicio 1:

a)

$G = (N, T, S, P)$

$N = \{ \langle \text{switch} \rangle, \langle \text{cases} \rangle, \langle \text{case} \rangle, \langle \text{default} \rangle, , \langle \text{bloque} \rangle, \langle \text{expresion} \rangle, \langle \text{literal} \rangle, \langle \text{id} \rangle, \langle \text{letra} \rangle, \langle \text{digito} \rangle \}$

$T = \{ \text{"switch"}, \text{"case"}, \text{"default"}, \text{"break"}, \text{"("}, \text{")"}, \text{"{"},$

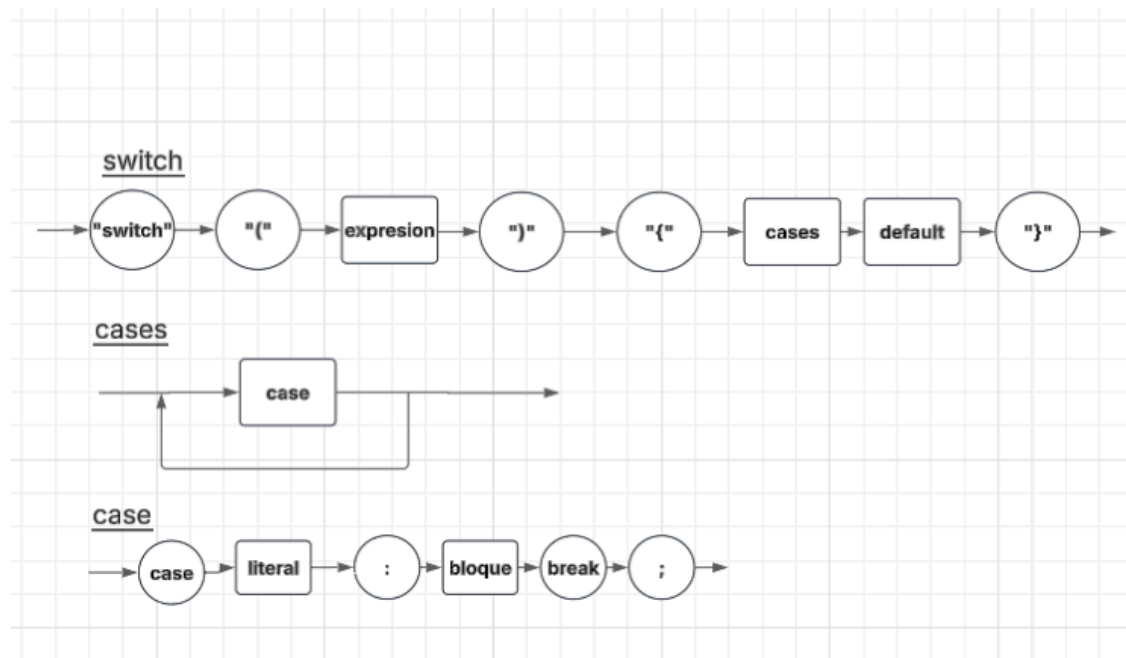
```

"}", ":", ";",
    "a".."z", "A".."Z", "0".."9" }
S = <switch>

P = {
<switch> ::= "switch" "(" <expresion> ")" "{" <cases> <default> "}"
<cases> ::= <case> | <case> <cases>
<case> ::= "case" <literal> ":" <bloque> "break" ";"
<default> ::= "default" ":" <bloque>
<bloque> ::= <id> ";" | <id> ";" <bloque>
<expresion> ::= <id> | <literal>
<literal> ::= <digito> | <digito> <literal> | <letra> | <letra> <literal>
<id> ::= <letra> | <letra> <id>
<letra> ::= "a" | ... | "z" | "A" | ... | "Z"
<digito> ::= "0" | ... | "9"
}

```

b)



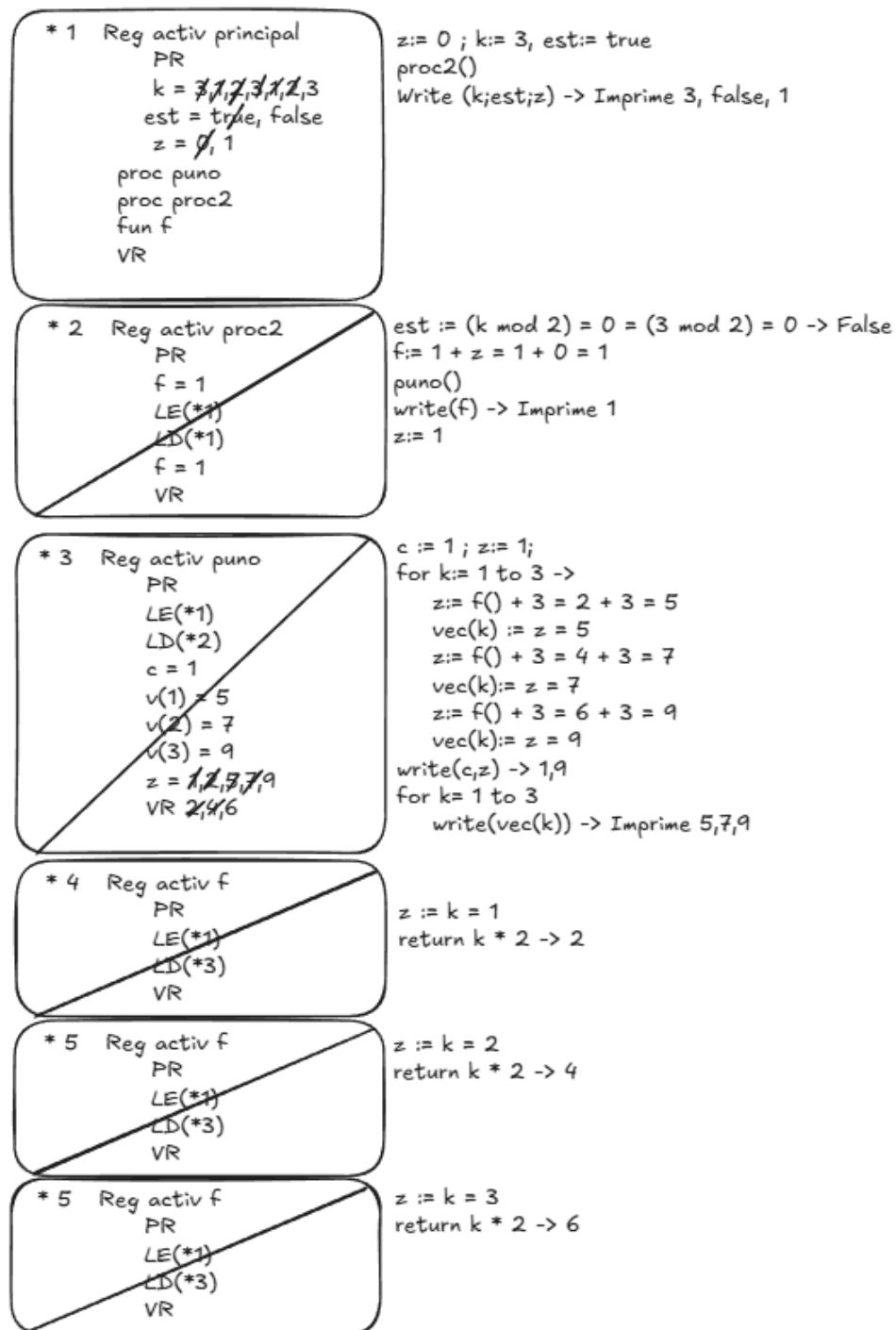
Me quede sin prueba gratis xd

Ejercicio 2:

Ident.	Tipo	r-valor	Alcance	T.V.
Main(2)			2-12	2-12
c (4)	automatica	basura	3-12	2-12
o (4)	automatica	basura	3-12	2-12
r (4)	automatica	basura	4-7.3 → 8-12	2-12
vec (5)	automatica	basura	6-12	2-12
c3 (6)	automatica	basura	7-12	2-12
Proc(7)			7.1-7.5	7-7.5
vec2	semidinamica	basura	7.3-7.5	7-7.5
r	automatica	nil	7.4-7.5	7-7.5
r^	dinamica	basura	7.4-7.5	7.4.3-7.4.4

Ejercicio 3:

a)



- En R.A de proc 2 falta LE(*1) y LD(*1) pero se entiende
- b) Los campos que componen un registro de activacion son:
- Registro de Activacion → Valores y atributos de un prodecimiento, main o funcion en forma de pla
 - Punto de retorno → Guarda la direccion de memoria donde la ejecucion continua luego de llamar a una rutina

- Link estatico → Es un enlace al registro donde estaticamente esta contenido el registro de activacion
- Link dinamico → Es un enlace al registro que dinamicamente llamo o invoca al registro de activacion
- Variables → Son variables del reg. activacion que pueden ir cambiando de valor a lo largo de la ejecucion del programa
- Parametros → Valores de los parametros al momento de ser invocado el reg activacion
- Procedures → Son procedimientos declarados dentro del bloque del reg. activacion
- Funciones → Funciones declaradas dentro del bloque del reg. activacion
- Valor de Retorno → Se almacena el valor del llamado a una funcion dentro de la logica del bloque una vez esta termina. Este valor tiene que ser almacenado aqui ya que de otra manera se perderia y es en el registro de activacion en donde es llamada esa funcion que se almacenaria

Ejercicio 4:

- a) Indique donde se almacenan las variables estaticas en C y cual es la politica de inicializacion de variables en C para variables globales y estaticas
 - Las variables estaticas en C se almacenan en memoria al momento de la compilacion al declararlas como 'static' y mantienen su valor en los diferentes llamados a funciones. La politica de inicializacion de variables en C puede ser por defecto, es decir `int x;` y tendria 0 ya que C inicializa por defecto sus variables estaticas o globales enteros en 0 o tambien puede ser `int x = 10` inicializandola con un valor al momento de la declaracion.
- b) Indique un ejemplo de variable donde su tiempo de vida sea mayor que su alcance. El ejemplo debe ser claro de acuerdo a lo visto en la explicacion en cuanto a la alocaion en memoria, no solo considerando los nujmeros de linea en el codigo. Puede ser en pseudocodigo (emulando pscal like, por ejemplo)
 - Un ejemplo donde el T.V de una variable sea mayor que su alcance es cuando esta se encuentra enmascarada, es decir, se declara al principio una variable con id "p" y luego dentro de una funcion vuelve a ser declarada "p" con el mismo identificador, lo que limita al alcance ya que la 2da variable le 'quitaria' parte del mismo a la primera, pero su tiempo de vida no se veria afectado.

Ejemplo en pascal-like:

```
1. Program main;  
  
2. int p; String hola;  
3. p:= 1  
4. Procedure hola() begin  
5.     int pepe; int p;  
6.     p := 10  
7.     p := 10- 1  
8. end;  
9 hola = "hola"  
end.
```

El alcance de p seria de la linea 2-4 y luego de la linea 8 en adelante ya que en el medio aparece p denuevo que tiene alcance de la linea 5-7. Sin embargo, su tiempo de vida es de la linea 2-9 lo cual es mayor a su alcance

c) Sea la siguiente afirmacion "un lenguaje interpretado solo detecta errores semanticos, ya que se chequea todo en ejecucion". Indique si la misma es verdadera o no, dando una justificacion apropiada

-La afirmacion es falsa, que sea interpretado no delimita el tipo de error que chequea. Un lenguaje interpretado puede detctar tantos errores semanticos en tiempo de ejecucion como errores isntacticos antes de ejecutarlo, por ejemplo, al momento de parsear

1er fecha 2023

Ejercicio 1

a) (Pts.15) Realice en EBNF la gramática de una de URL teniendo en cuenta el siguiente formato (para el protocolo http únicamente y no considerar direcciones ip):

protocolo://servicio.nombreDominio/rutaEnElServidor/pagina.htm

Ejemplos: <https://unlp.edu.ar/enseñanza/posgrado/docentes-6508-11508/>

b) (Pts. 5) Realice el árbol de derivación de : <https://www.lanacion.com.ar/cultura/feria-del-libro-nid26042023/>

Ejercicio 2

Sea el siguiente código en Ada, indique para todos los identificadores

a) (Pts. 5) Su tipo de ligadura con l-valor. b) (Pts. 5) Su r-valor al momento de declaración (inicialización).

c) (Pts. 5) Tiempo de vida . d) (Pts. 5).- alcance

```

1. with text_io; use text_io;
2. Procedure Main is;
3.   type arreglo is array(integer range <>);
4.   a, n, p:integer;
5.   vec1:arreglo(1..100);
6.   c: constant integer:=10;
7.   Procedure Uno is;
8.     7.1.   type puntero is access integer;
9.     7.2.   vec2:arreglo(0..c);
10.    7.3.   p: puntero;
11.    7.4.   begin
12.      7.4.1.   n:=4;
13.      7.4.2.   vec2(n):= vec2(1) +
14.      7.4.3.   p:= new puntero;
15.      7.4.4.   free p;
16.    7.5.   end;
17. begin
18.   n:=5;
19.   Uno;
20.   a:= n + 2;
21. end.

```

Realice este ejercicio sobre esta misma hoja.

Identif	L-value	R-Value	Alcance	T.V.
main(2)	—	—	2-12	2-12
a(4)	autom.	basura	4-12	2-12
n(4)	autom.	basura	4-12	2-12
p(4)	autom.	basura	4-7.2 8-12	2-12
vec1(5)	autom.	basura	5-12	2-12
c(6)	autom.	basura 10	6-12	2-12
vec2(7)	—	—	7-12	7-7.5
vec2(7.2)	autom.	basura	7.2-7.5	7-7.5
p(7.3)	autom.	null	7.3-7.5	7-7.5
^p	dimam	basura	7.3-7.5	7.4.3-7.4.4

→ c constante común, live en memoria!

Ejercicio 3

Sea el siguiente programa escrito en Pascal like, realice la ejecución del programa presentado siguiendo:
a) (Pts. 15) Cadena dinámica. b) (Pts. 15) Cadena estática.

<pre>Program principal Var i:integer; Var estado:boolean; var z: integer; Procedure puno; Var a: integer; Var vec: array[1..4] of integer; Var z: integer; Begin a:=1; z:=1; for i:=1 to 4 do begin z:= f+3; vec[i]:=z; end; if (estado = true) then begin a:=z; end; write(a, z); For i=1 to 4 begin write(vec(i)); end; End;</pre>	<pre>Procedure proc2; var f: integer; begin estado := (i mod 2) =0; f:=1+z; if (z==0) puno; write(f); z:=1; end; Función f:integer; Begin z:= i; Return i*3; End; Begin f z:=0; i:=3; estado:=true; proc2(); Write(i;estado;z); END.</pre>
--	--

Ejercicio 4

- a) (Pts. 10) Indique porqué es importante contar con criterios de evaluación para los lenguajes. Detalle dos que le parezca relevante que un lenguaje posea y explique por qué.
- b) (Pts. 10) Constante y variable estática. Analice los atributos asociados en ambos conceptos, indicando momentos de ligadura y estabilidad. De acuerdo al análisis realizado indique si son conceptos similares o no. Justifique.
- c) (Pts. 10) Diga que analiza la semántica estática y semántica dinámica. Ejemplifique.

Ejercicio 1)

a)

```
G = (N,T,S,P)
N = { <url>, <protocolo>, <servicio>, <dominio>, <ruta>, <pagina>,
      <letra>, <digito>, <palabra>
T = { 'https', 'http', 'a .. z', 'A .. Z', '0 .. 9', '.', '/',
      '-', '.html', '://' }
S = <url>
P = {
      <url> ::= <protocolo>'://'<servicio> '.'<dominio>{'/'<ruta>
      >}* '/'<pagina>
      <protocolo> ::= ('https'|'http')
```

```

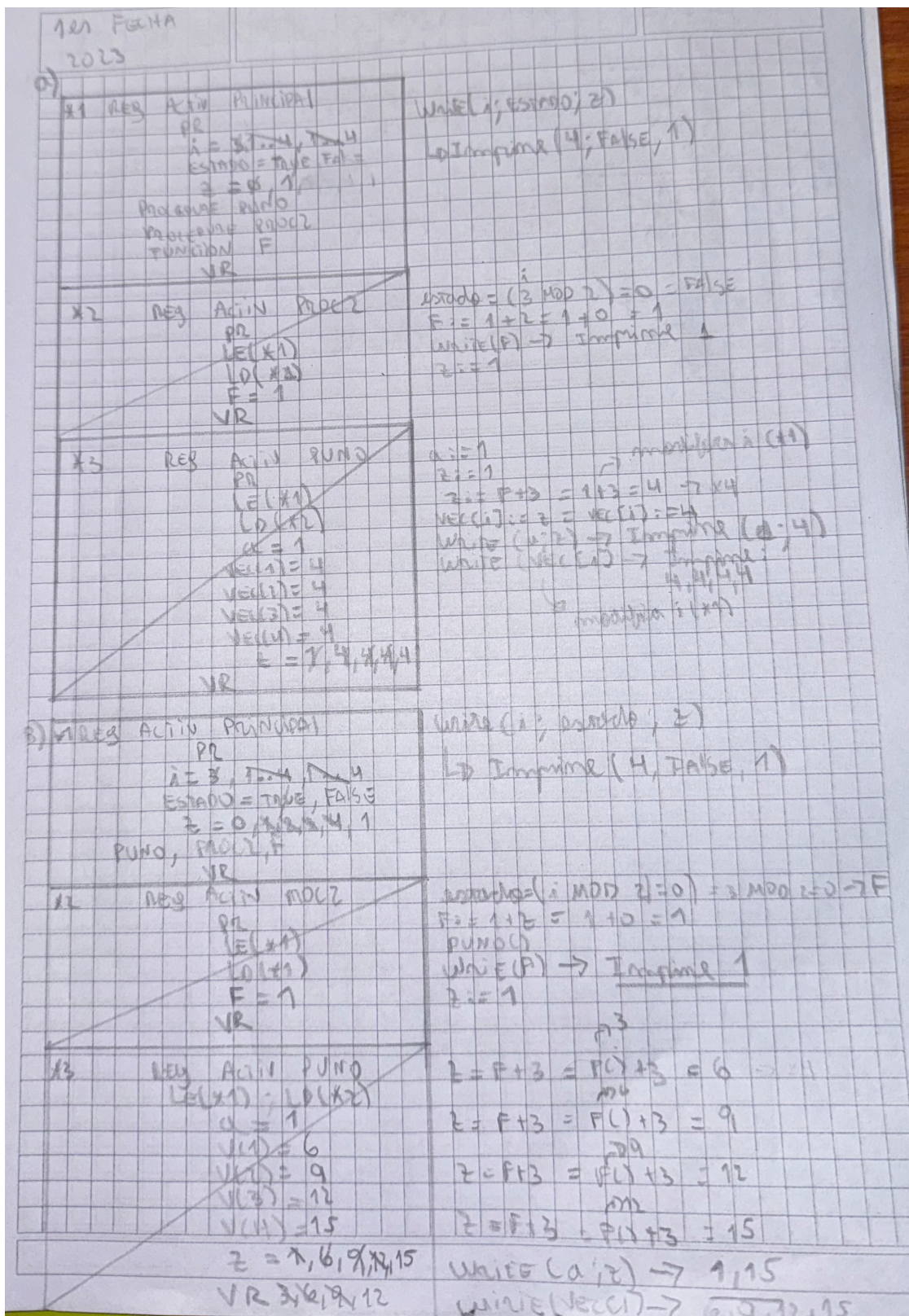
<servicio> ::= ['www' '.'.]<ruta>
<dominio> ::= <palabra> {'.'<palabra>}*
<ruta> ::= {(<letra>|<digito>)'-' }+
<pagina> ::= <letra> {(<letra>|<digito>)}* ['.html'] ['/' ]
<palabra> ::= {<letra>}+
<letra> ::= (a|..|z|A|..|Z)
<digito> ::= (0|..|9)
}

```

Ejercicio 2:

Ident.	Tipo	r-valor	Alcance	T.V.
Main (2)			3-12	2-12
a (4)	automatica	basura	5-12	2-12
n (4)	automatica	basura	5-12	2-12
p (4)	automatica	basura	5-7.3 → 8-12	2-12
vec1 (5)	automatica	basura	6-12	2-12
c (6)	automatica	basura	7-12	2-12
Uno (7)			7.1-7.5	7-7.5
vec2 (7.2)	semidinamica	basura	7.3-7.5	7-7.5
p (7.3)	automatica	nil	7.4-7.5	7-.7.5
p^	dinamica	basura	7.4-7.5	7.4.3-7.4.4

Ejercicio 3:



-En *2 me falto poner $f = 1$

X4	REG	ACT	N	F			$Z := \bar{A} = 1(11) \rightarrow \text{modificar el } \bar{A} \text{ de } (11)$ $\text{RETURN } \bar{A}K3 = 3$
					LE(11)		
					LD(13)		
					VR		
X5	REG	ACT	N	F			$Z := \bar{A} = 2 \rightarrow (11)$ $\text{RETURN } \bar{A}K3 = 6$
					LE(11)		
					LD(13)		
					VR		
X6	REG	ACT	N	F			$Z := \bar{A} = 3$ $\text{RETURN } \bar{A}K3 = 9$
					LE(11)		
					LD(13)		
					VR		
X7	REG	ACT	N	F			$Z := \bar{A} = 4$ $\text{RETURN } \bar{A}K4 = 12$
					LE(11)		
					LD(13)		
					VR		

Ejercicio 4:

a) Indique por que es importante contar con criterios de evaluacion para los lenguajes. Detalle dos que le parezca relevante que un lenguaje posea y explique por que

-Es importante contar con criterios de evaluacion para los lenguajes ya que nos permite determinar si el lenguaje es apto o no para ser considerado bueno y poder ser utilizado en masa. Uno de los atributos mas importantes es la legibilidad y simplicidad, ya que permite una facil escritura y lectura del codigo y a futuro permite que sea facil de masterizar y aprender. Otro importante podria ser la confiabilidad, ya que controla que el programa no se rompa con chequeo de tipos y manejo de excepciones

b) Constante y variable estatica. Analice los atributos asociados en ambos conceptos, indicando momento de ligadura y estabilidad. De acuerdo al analisis realizado indique si son conceptos similares o no. Justifique

-Una constante es un atributo cuyo valor no cambia durante la ejecucion del programa, su momento de ligadura ocurre en compilacion y su valor se conoce antes de ejecutar el programa, por lo que se mantiene estable durante la ejecucion del mismo. Por otro lado estan las variables estaticas, las cuales son aquellas que mantienen su valor entre distintas ejecuciones (se liga en compilacion) de un bloque pero si pueden cambiar.

En cuanto a momento de ligadura si son similares ya que ambas se ligan en momento de compilacion y existen durante toda la ejecucion, sin embargo son diferentes durante la ejecucion ya que las constantes no pueden ser modificadas y las variables estaticas si pueden serlo, permitiendo mas versatilidad

c) Diga que analiza la semantica estatica y la semantica dinamica. Ejemplifique

-La semantica estatica analiza el programa antes de ejecutarse, enfocandose en tipos de datos, declaraciones y compatibilidad. Por otro lado la semantica dinamica se enfoca mas en la ejecucion del programa, teniendo en cuenta valores reales y errores en tiempo de ejecucion

Un ejemplo de semantica estatica es: `int x = 10, x = x + "Hola"` se detecta en compilacion ya que no le puedes sumar un string a un tipo de dato integer. En cambio un ejemplo de semantica dinamica es `vec = array[1..10] of integer, vec[11] = 60`, se detecta en ejecucion ya que estarias accediendo a una posicion del vector que esta fuera de su rango